

Pieter Ayal

pietera.mullock@gmail.com | linkedin.com/in/pieterayal | github.com/pieter124

EDUCATION

University of Manchester

Manchester, United Kingdom

Bachelor of Science (Honours) in Computer Science with Industrial Experience

Sep. 2024 – Jun. 2028

- Academic Performance: First Year: **75.3%** (First Class) | Second Year: **76.8%** (First Class)
- Relevant Coursework: Data Structures and Algorithms, Compilers, System Architecture, Concurrent Programming, Distributed Systems, Operating Systems, Software Engineering

EXPERIENCE

SLB

Abingdon, United Kingdom

HPC Software Engineering Intern

Sep. 2026 – Apr. 2027

- Incoming placement focused on **upscaling embedded meshes** for HPC simulation workflows.

Thought Machine

London, United Kingdom

Software Engineering Intern

Jun. 2026 – Sep. 2026

- Architected and developed a full-stack **Balances Visualiser** in **TypeScript** and **React** to accelerate incident debugging for on-call engineers and simplify onboarding onto the ledger's complex accounting model.
- Owned the project end-to-end in a production engineering environment, from initial design and RFCs through cross-team code reviews and technical reviews.
- Built a **JSON parser** to validate financial transaction payloads against **Vault Core's REST API** schema, catching malformed or non-conformant data.

PROJECTS

Fly.io Gossip Glomers | Go | github.com/pieter124/fly.io-gossip-glomers

- Implemented a fault-tolerant broadcast protocol using a fanout-tree topology (branching factor 4) with per-neighbor batching and retry-until-acknowledged delivery, verified on Fly.io's **Maelstrom** (Jepsen-based) test harness.
- Built a partitioned **Kafka-style commit log** with hash-based key ownership and RPC forwarding across nodes, verified via Maelstrom's Jepsen checker across **18,621** operations with **zero failures** and **99.97%** availability (1.24 inter-server msgs/op).

Concurrent Bank Ledger | Go, gRPC, Protobuf | github.com/pieter124/concurrentbankledger

- Built a thread-safe, in-memory banking engine in Go, exposing gRPC endpoints utilizing Protobuf for account initialization and double-entry transfers.
- Designed and benchmarked three concurrency strategies: fine-grained per-account mutexes, a single-actor model, and a lock-free sharded-actor architecture.
- Implemented a two-phase reserve/credit/finalize protocol for cross-shard transfers, ensuring zero-sum conservation under concurrent load.
- Reduced per-transfer latency by **2x** versus the single-actor model and **~25%** versus fine-grained mutexes (1,090 ns/op) by sharding work across N actor goroutines.
- Engineered a write-ahead log (append + fsync, JSON records) to provide crash recovery, idempotency, and state rebuilding on restart.

EXTRA-CURRICULAR ACTIVITIES

Google Student AI Hackathon 2026 | 3rd Place | github.com/pieter124/google-ai-hackathon-2026

Hackabot (Quansar Track) 2026 | 2nd Place | github.com/pieter124/hackabot26

ICHack 2026 | Participation | github.com/pieter124/ICHack26

DurHack 2025 | Participation | github.com/itzbenjamin17/OctoBoard

SAT Solver Competition 2025 | 2nd Place | github.com/pieter124/rustSAT

EPCC HPC Summer School 2025 | Student

TECHNICAL SKILLS

Languages: TypeScript, Go, Python, Rust, C

Frameworks & APIs: React, gRPC, REST

Parallelism: OpenMP, MPI, SLURM

Tools: Protobuf, Git, CMake